

# Using distance to improve implicit state classification of Markov chains

Andrew Miner

Shuxing Cheng

Department of Computer Science

Iowa State University, Ames, Iowa, 50011

E-mail: {asminer, scheng}@cs.iastate.edu

## 1. Introduction

Markov Chains are a well-established and important tool to assess the performance and dependability of computer and communication systems. An important step in many types of Markov chain analysis is the classification of states into recurrent classes and the class of transient states [11]. For instance, the stationary distribution of a Markov chain is computed by analyzing the recurrent classes in isolation, and determining the probability that the Markov chain reaches each recurrent class. Recurrent classes are traditionally determined using well-known algorithms to compute strongly-connected components (SCCs) [1]: a SCC that cannot reach states outside of the SCC is a recurrent class; all others are transient states.

A major hurdle is that the number of states in a Markov chain can be extremely large for practical systems. This state-explosion problem effectively limits the applicability of *explicit* techniques, which require to explicitly visit and represent each state of the Markov chain. As a result, many researchers have turned instead to *implicit* (also known as *symbolic*) techniques, which exploit structural characteristics of the system. Such techniques include representing the states of the Markov chain using binary decision diagrams (BDDs) [12] or multi-way decision diagrams (MDDs) [4], and representing the transition matrix of the Markov chain using Kronecker algebra [3], multi-terminal BDDs [8], or matrix diagrams [5].

Existing implicit techniques to classify Markov chain states include those based on transitive closure [7], which are typically quite computationally expensive; those based on lockstep search [2], which look only one step forward and backward at a time; and those based on forward and backward reachability [12]. Since recent advancements [4] have led to greatly improved forward and backward reachability, we focus our attention on this classification algorithm. However, as presented, the algorithm in [12] requires to “randomly select” a state at each step of the iteration; in this extended abstract, we suggest the use of distance information as a heuristic to select states in a smarter way.

The remainder of the extended abstract is organized as follows. Section 2 reviews background and related work in more depth. Section 3 reviews the classification algorithm in [12] and describes our distance-based heuristic for improving the algorithm. Section 4 discusses various implementation issues and possible directions for future work.

## 2. Background and related work

### 2.1. Kronecker representations of Markov chains

We consider finite Markov chains constructed from a high-level, structured formalism with  $K$  components, and set of events  $\mathcal{E}$ . Each component  $k$  consists of local states  $\mathcal{S}_k = \{0, \dots, n_{k-1}\}$ , and the set of potential states of the overall model is  $\hat{\mathcal{S}} = \mathcal{S}_K \times \dots \times \mathcal{S}_1$ . Thus, a state can be written  $\mathbf{i} = (i_K, \dots, i_1)$ . We assume that the contribution to the transition matrix for the potential Markov chain (i.e., the Markov chain with state space  $\hat{\mathcal{S}}$ ) due to a single event  $e \in \mathcal{E}$  can be expressed as the Kronecker product of contributions local to each component:  $\hat{\mathbf{R}}^e = \mathbf{W}_K^e \otimes \dots \otimes \mathbf{W}_1^e$ . The overall transition matrix  $\hat{\mathbf{R}}$  is then the sum of the  $\hat{\mathbf{R}}^e$  matrices over all events [3]. Note that we require the matrix  $\hat{\mathbf{R}}$  to contain zeroes along the diagonal (to efficiently handle absorbing states).

### 2.2. Encoding sets of states with MDDs

Multi-way decision diagrams (MDDs) [9] can be used to encode any subset of  $\hat{\mathcal{S}}$  [4]. Formally, an MDD is a directed, acyclic graph with integer-labeled terminal nodes, and non-terminal nodes labeled with some component  $k$  and containing  $n_k$  pointers to other nodes. A state  $(i_K, \dots, i_1)$  is contained in the set encoded by the MDD if the path through the MDD, starting at the root node and following pointer  $i_k$  at a non-terminal node for component  $k$ , reaches terminal node 1. By applying appropriate reduction rules [9], MDDs can often represent large sets of states in a very compact way (i.e., with significantly fewer nodes than the number of

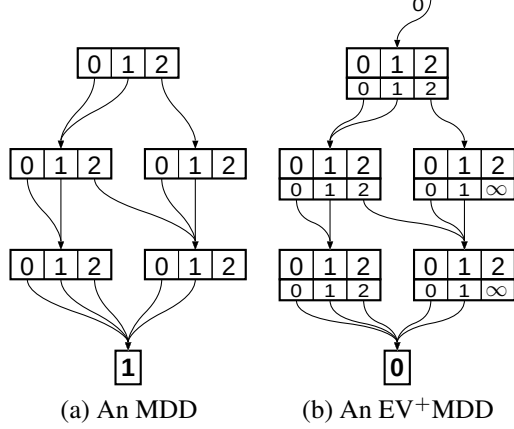


Figure 1. Examples of MDDs and EV<sup>+</sup>MDDs

states), although their worst-case behavior is at least a factor of  $K$  worse than explicit storage. Algorithms to manipulate MDD-encoded sets (e.g., set union, intersection, etc.) typically have complexity that depends upon the number of MDD nodes, rather than the sizes of the encoded sets, and thus can be quite efficient. An example MDD is shown in Figure 1(a), where  $K = 3$ , and  $\mathcal{S}_3 = \mathcal{S}_2 = \mathcal{S}_1 = \{0, 1, 2\}$ . Pointers to terminal node 0 are omitted for clarity. The MDD encodes the set  $\mathcal{S} \subset \mathcal{S}_3 \times \mathcal{S}_2 \times \mathcal{S}_1$  where at most one component is in local state 2.

We will also utilize edge-valued MDDs (EV<sup>+</sup>MDDs) as described in [6]. EV<sup>+</sup>MDDs are similar to MDDs, but assign an edge value (either infinity or a natural) to each pointer, and have a single terminal node, labeled with 0. An EV<sup>+</sup>MDD is used to encode a function  $f : \hat{\mathcal{S}} \rightarrow \mathbb{N} \cup \{\infty\}$ : by following the path for a state  $\mathbf{i} = (i_K, \dots, i_1)$ , the value of  $f(\mathbf{i})$  is found by summing the edge values encountered until terminal node 0 is reached. Since any information after an infinite edge value is irrelevant, by convention such edges are always directly to terminal node 0. Additionally, each node must contain an edge value of zero. Reduction rules for EV<sup>+</sup>MDDs are discussed in [6]. An example EV<sup>+</sup>MDD is shown in Figure 1(b). Pointers with edge value of infinity (which are directly to terminal node 0) are omitted for clarity. The EV<sup>+</sup>MDD encodes the function  $f$  where  $f(\mathbf{i}) = i_3 + i_2 + i_1$  if  $\mathbf{i} \in \mathcal{S}$ ,  $f(\mathbf{i}) = \infty$  otherwise.

### 2.3. Generating forward and backward sets

We define functions for one-step *forward* reachability from a state,  $F(\hat{\mathbf{R}}, \mathbf{i}) = \{\mathbf{j} : \hat{\mathbf{R}}[\mathbf{i}, \mathbf{j}] > 0\}$ , and one-step *backward* reachability from a state,  $B(\hat{\mathbf{R}}, \mathbf{j}) = \{\mathbf{i} : \hat{\mathbf{R}}[\mathbf{i}, \mathbf{j}] > 0\}$ . We extend this to include reachability from sets of states,  $F(\hat{\mathbf{R}}, \mathcal{S}) = \bigcup_{\mathbf{i} \in \mathcal{S}} F(\hat{\mathbf{R}}, \mathbf{i})$  and  $B(\hat{\mathbf{R}}, \mathcal{S}) = \bigcup_{\mathbf{j} \in \mathcal{S}} B(\hat{\mathbf{R}}, \mathbf{j})$ ; and also to the notion of  $n$ -step or fewer for-

ward and backward reachability from a state:

$$\begin{aligned} F^0(\hat{\mathbf{R}}, \mathbf{i}) &= \{\mathbf{i}\} & F^{n+1}(\hat{\mathbf{R}}, \mathbf{i}) &= F^n(\hat{\mathbf{R}}, \mathbf{i}) \cup F(F^n(\hat{\mathbf{R}}, \mathbf{i})) \\ B^0(\hat{\mathbf{R}}, \mathbf{j}) &= \{\mathbf{j}\} & B^{n+1}(\hat{\mathbf{R}}, \mathbf{j}) &= B^n(\hat{\mathbf{R}}, \mathbf{j}) \cup B(B^n(\hat{\mathbf{R}}, \mathbf{j})) \end{aligned}$$

with similar definitions for sets of states. Since  $\hat{\mathcal{S}}$  is finite, the limits  $F^*(\hat{\mathbf{R}}, \mathbf{i}) = \lim_{n \rightarrow \infty} F^n(\hat{\mathbf{R}}, \mathbf{i})$  and  $B^*(\hat{\mathbf{R}}, \mathbf{j}) = \lim_{n \rightarrow \infty} B^n(\hat{\mathbf{R}}, \mathbf{j})$  exist and are also finite. Note that  $F^*(\hat{\mathbf{R}}, \mathbf{i})$  is the set of all states that can be reached from state  $\mathbf{i}$ , and  $B^*(\hat{\mathbf{R}}, \mathbf{j})$  is the set of all states that can reach state  $\mathbf{j}$ .

Recent advancements have made it possible to compute the sets  $F^*(\hat{\mathbf{R}}, \mathcal{S}^0)$  and  $B^*(\hat{\mathbf{R}}, \mathcal{S}^0)$  efficiently for many systems, using MDD-based algorithms. In particular, for highly asynchronous systems (characterized by many identity matrices in the Kronecker representation for  $\hat{\mathbf{R}}$ ), the *saturation* algorithm [4] is able to exploit the asynchronous behavior to generate extremely large sets  $F^*(\hat{\mathbf{R}}, \mathcal{S}^0)$  and  $B^*(\hat{\mathbf{R}}, \mathcal{S}^0)$  very efficiently, using a Kronecker representation for  $\hat{\mathbf{R}}$ . Furthermore, a variant of saturation [6] can be used to construct the *distance* function  $d_i$ , which indicates the minimum number of forward steps required to reach each state from state  $\mathbf{i}$  (or set of states)

$$d_i(\hat{\mathbf{R}}, \mathbf{j}) = \begin{cases} \min\{n : \mathbf{j} \in F^n(\hat{\mathbf{R}}, \mathbf{i})\} & \text{if } \mathbf{j} \in F^*(\hat{\mathbf{R}}, \mathbf{i}) \\ \infty & \text{if } \mathbf{j} \notin F^*(\hat{\mathbf{R}}, \mathbf{i}) \end{cases}$$

using EV<sup>+</sup>MDDs. Note that the EV<sup>+</sup>MDD in Figure 1(b) encodes the distances from the initial state  $(0, 0, 0)$ , where each local component may transition from local state 0 to 1 and from 1 to 2 if no other component is in state 2.

## 3. State classification

Given an initial set of states  $\mathcal{S}^0$  and a Kronecker representation for a potential CTMC  $\hat{\mathbf{R}}$ , we can determine the *actual* CTMC, which has state space  $\mathcal{S} = F^*(\hat{\mathbf{R}}, \mathcal{S}^0)$ , using the saturation algorithm [4]. The problem we consider is how to classify the states of the actual CTMC into recurrent classes and transient states.

### 3.1. Previous work

The classification algorithm of Xie and Beerel [12], which we use for our approach, is based on the following properties:

- A state  $\mathbf{i} \in \mathcal{S}$  is recurrent if and only if  $F^*(\hat{\mathbf{R}}, \mathbf{i}) \subseteq B^*(\hat{\mathbf{R}}, \mathbf{i})$ .
- A state  $\mathbf{i} \in \mathcal{S}$  is transient if and only if  $F^*(\hat{\mathbf{R}}, \mathbf{i}) \not\subseteq B^*(\hat{\mathbf{R}}, \mathbf{i})$ .

```

Classify(in: MDD  $\mathcal{S}$ , CTMC  $\hat{\mathbf{R}}$ )
1:  $\mathcal{S}' \leftarrow \mathcal{S}$ ;
2: while  $\mathcal{S}' \neq \emptyset$  do
3:    $\mathbf{i} \leftarrow \text{RandomTake}(\mathcal{S}')$ ;
4:    $\mathcal{F} \leftarrow F^*(\hat{\mathbf{R}}, \mathbf{i})$ ;
5:    $\mathcal{B} \leftarrow B^*(\hat{\mathbf{R}}, \mathbf{i})$ ;
6:   if  $\mathcal{F} \subseteq \mathcal{B}$  then           •  $\mathbf{i}$  is recurrent
7:      $\mathcal{F}$  is a recurrence class;
8:      $\mathcal{B} \setminus \mathcal{F}$  are transient states;
9:   else                       •  $\mathbf{i}$  is transient
10:     $\mathcal{B}$  are transient states;
11:   end if
12:    $\mathcal{S}' \leftarrow \mathcal{S}' \setminus \mathcal{B}$ ;
13: end while

```

**Figure 2. Xie and Beerel’s classification algorithm [12].**

- If state  $\mathbf{i}$  is recurrent, the set  $F^*(\hat{\mathbf{R}}, \mathbf{i})$  is a recurrent class, and all states in the set  $B^*(\hat{\mathbf{R}}, \mathbf{i}) \setminus F^*(\hat{\mathbf{R}}, \mathbf{i})$  are transient.
- If state  $\mathbf{i}$  is transient, then all states in the set  $B^*(\hat{\mathbf{R}}, \mathbf{i})$  are also transient.

Based on the above properties, they derive the classification algorithm shown in Figure 2, which repeatedly picks an unclassified state at random, determines if it is transient or recurrent, and classifies all unclassified states that can reach the selected state (including the selected state itself). For each iteration of the algorithm, it is necessary to compute the forward and backward sets for the selected state. The number of iterations required depends on the size of  $\mathcal{B} \cap \hat{\mathcal{S}}$ , which depends on the selected state and structure of the Markov chain. In the best case, we are very lucky, and the randomly selected states are always recurrent, causing the algorithm to complete after exactly  $C$  iterations, where  $C$  is the number of recurrent classes (this can be seen by noticing that every transient state can reach at least one recurrent class). In the worst case, we are very unlucky, and  $\mathcal{B} \cap \hat{\mathcal{S}}$  contains only the selected state; if the recurrent classes contain only one state, the algorithm can require  $|\mathcal{S}|$  iterations.

### 3.2. Our improvements

An immediate improvement to Xie and Beerel’s classification algorithm is to use saturation [4] to compute the forward and backward sets. The “traditional” iterative algorithms for forward and backward reachability, which are used in [12] and throughout the literature, can be orders of magnitude slower than the saturation algorithm [4], especially for asynchronous systems. Another simple improvement is the classification of absorbing states: any potential

```

DistanceClassify(in: MDD  $\mathcal{S}$ , EV+MDD  $\mathcal{D}$ , CTMC  $\hat{\mathbf{R}}$ )
•  $\mathcal{D}$  is the EV+MDD encoding of  $d_{\mathcal{I}}(\hat{\mathbf{R}}, \mathbf{j})$ ,
• for initial states  $\mathcal{I}$ , for all  $\mathbf{j} \in \hat{\mathcal{S}}$ .
1:  $\mathcal{S}' \leftarrow \mathcal{S}$ ;
2: while  $\mathcal{S}' \neq \emptyset$  do
3:    $\mathbf{i} \leftarrow \text{TakeLargest}(\mathcal{D})$ ;
4:    $\mathcal{F} \leftarrow F^*(\hat{\mathbf{R}}, \mathbf{i})$ ;
5:    $\mathcal{B} \leftarrow B^*(\hat{\mathbf{R}}, \mathbf{i})$ ;
6:   if  $\mathcal{F} \subseteq \mathcal{B}$  then           •  $\mathbf{i}$  is recurrent
7:      $\mathcal{F}$  is a recurrence class;
8:      $\mathcal{B} \setminus \mathcal{F}$  are transient states;
9:   else                       •  $\mathbf{i}$  is transient
10:     $\mathcal{B}$  are transient states;
11:   end if
12:    $\mathcal{S}' \leftarrow \mathcal{S}' \setminus \mathcal{B}$ ;
13:   RemoveStates( $\mathcal{D}$ ,  $\mathcal{B}$ );
14: end while

```

**Figure 3. Distance-based classification**

state  $\mathbf{i}$  with an empty row in the matrix  $\hat{\mathbf{R}}$  is an absorbing state, and thus can be immediately classified as its own recurrent class. An MDD encoding the set of empty rows of  $\hat{\mathbf{R}}$  can be efficiently constructed based on the Kronecker representation of  $\hat{\mathbf{R}}$  [10]. Thus, absorbing states can be discovered and classified in a preprocessing step.

Our core contribution is to consider alternatives to random selection of states to explore. In an ideal situation, the algorithm should always select a recurrent state; failing that, the algorithm should try to select a state with a large number of unclassified states in its backward set, since the algorithm classifies states in the backward set at each iteration. We propose a simple heuristic for selecting states based on distance information: *always select an unclassified state with furthest distance from the initial state(s)*. The above rule, while simple, has an intuitive explanation: it attempts to select a state with a large backward set using only distance information. Indeed, if a state  $\mathbf{j}$  has distance  $\delta$  from the initial states, then there is a path of length  $1 + \delta$  from an initial state to state  $\mathbf{j}$ , and all the states in the path will belong to the backward set of  $\mathbf{j}$ . Of course, this simple rule is not guaranteed to select the state with largest backward set; it only guarantees that the backward set contains  $1 + \delta$  states. Examples can be contrived where the heuristic does not perform well; however, we believe that for many types of systems, the heuristic is appropriate. In particular, for systems that degrade gracefully (i.e., the system continues to provide as much service as possible in the presence of failures), the underlying CTMC will contain several SCCs, each corresponding to various levels of service, and the recurrent states are likely to be “far” from the initial states.

Our distance-based classification algorithm is shown in Figure 3 for comparison. We assume that the EV<sup>+</sup>MDD encoding the state distances from the initial states has al-

ready been constructed as described in [6]. Procedure `TakeLargest` finds one of the states with largest finite distance in  $\mathcal{D}$ , and procedure `RemoveStates` is used to set the distance information of the newly classified states to infinity (since they should not be considered for selection in the future).

## 4. Discussion

We are currently developing an implementation of the distance-based classification algorithm. Note that this requires efficient algorithms for procedures `TakeLargest` and `RemoveStates`. It is interesting that, while finding a state with minimum distance is easy (at each node select a pointer with edge value of zero) [6], finding a state with maximum finite distance (procedure `TakeLargest`) requires a bit more work: from the bottom up, visit each node and determine the path with maximum finite distance, based upon the edge values and the maximum distance at the nodes below. Procedure `RemoveStates` is essentially a variation of set difference, with care taken to adjust the edge values of the resulting  $EV^+MDD$  appropriately. Another way to implement `RemoveStates` is to use the `IntersectionMax` operator [6]: construct an  $EV^+MDD \mathcal{X}$  with distances of zero for the states to keep, and distances of infinity for the states to remove; then pass both the distance function  $\mathcal{D}$  and  $\mathcal{X}$  to `IntersectionMax`.

One concern with this approach is that construction of the  $EV^+MDD$  encoding the distances, even using saturation, may be expensive, and that modifying the distance  $EV^+MDD$  at each iteration may make the approach prohibitively expensive (since an  $EV^+MDD$  may require many more nodes than an MDD, due to differences in edge values). Alternatively, the overhead of maintaining the distances may be acceptable, and may drastically reduce the number of iterations required to classify the states. We intend to investigate this issue, in part by comparing the two algorithms experimentally (i.e., `Classify` versus `Distance-Classify`) on a variety of CTMCs constructed from high-level formalisms.

Other distance-based heuristics should be investigated. For instance, when the selected state  $i$  is determined to be transient, it might be better to choose the next state as one with maximum distance from state  $i$ , rather than from the initial state (with the hope that this will quickly lead to a recurrent state). Alternatively, at every step, it might be best to choose the next state as the one furthest from all states that have already been classified, with the hope that the number of unclassified states in the backward set is large. Clearly, heuristics such as these will have a higher overhead, as they require distance function construction at every iteration. However, this effort may be well-spent if the overall number of iterations required is reduced.

## References

- [1] A. Aho, J. Hopcroft, and J. Ullman. *Data Structures and Algorithms*. Addison-Wesley, 1983.
- [2] R. Bloem, H. Gabow, and F. Somenzi. An algorithm for strongly connected component analysis in  $n \log n$  symbolic steps. In *Proc. International Conference on Formal Methods in Computer-Aided Design (FMCAD)*, pages 37–54. Springer-Verlag, Nov. 2000.
- [3] P. Buchholz, G. Ciardo, S. Donatelli, and P. Kemper. Complexity of memory-efficient Kronecker operations with applications to the solution of Markov models. *INFORMS J. Comp.*, 12(3):203–222, SUMMER 2000.
- [4] G. Ciardo, G. Luetzgen, and R. Siminiceanu. Saturation: an efficient iteration strategy for symbolic state space generation. In T. Margaria and W. Yi, editors, *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 2031, pages 328–342, Genova, Italy, Apr. 2001. Springer-Verlag.
- [5] G. Ciardo and A. S. Miner. A data structure for the efficient Kronecker solution of GSPNs. In P. Buchholz, editor, *8th Int. Workshop on Petri Nets and Performance Models (PNPM'99)*, pages 22–31, Zaragoza, Spain, Sept. 1999. IEEE Comp. Soc. Press.
- [6] G. Ciardo and R. Siminiceanu. Using edge-valued decision diagrams for symbolic generation of shortest paths. In M. D. Aagaard and J. W. O'Leary, editors, *Proc. Fourth International Conference on Formal Methods in Computer-Aided Design (FMCAD)*, LNCS 2517, pages 256–273, Portland, OR, USA, Nov. 2002. Springer-Verlag.
- [7] G. Hachtel, E. Macii, A. Pardo, and F. Somenzi. Markovian analysis of large finite state machines. *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems*, 15(12):1479–1493, Dec. 1996.
- [8] H. Hermanns, J. Kayser, and M. Siegle. Multi terminal binary decision diagrams to represent and analyse continuous time Markov chains. In B. Plateau, W. J. Stewart, and M. Silva, editors, *Numerical Solution of Markov Chains*, pages 188–207, Zaragoza, Spain, June 1999. Prensas Universitarias de Zaragoza.
- [9] T. Kam, T. Villa, R. Brayton, and A. Sangiovanni-Vincentelli. Multi-valued decision diagrams: theory and applications. *Multiple-Valued Logic*, 4(1–2):9–62, 1998.
- [10] A. S. Miner and G. Ciardo. Efficient reachability set generation and storage using decision diagrams. In H. Kleijn and S. Donatelli, editors, *Application and Theory of Petri Nets 1999 (Proc. 20th Int. Conf. on Applications and Theory of Petri Nets)*, LNCS 1639, pages 6–25, Williamsburg, VA, USA, June 1999. Springer-Verlag.
- [11] W. J. Stewart. *Introduction to the Numerical Solution of Markov Chains*. Princeton University Press, 1994.
- [12] A. Xie and P. A. Beerel. Efficient state classification of finite state Markov chains. *IEEE Trans. Computer-Aided Design*, 17(12):1334–1339, 1998.